ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Разработка открытого инструмента статического анализа PE-файлов для выявления намеренных искажений структуры и заголовков

Студент	Научный руководитель	Кафедра и период
Колесник Дмитрий Сергеевич	Маркин Алексей Леонидович	806 «Вычислительная математика и программирование», 2026–2027г.

Источник проблемы и последствия

01 Кто источник файлов

Внешний трафик (вложения из почты, загрузки из браузера), зараженные файлы с флешек, подозрительные бинарные файлы, собранные с рабочих станций при расследовании инцидента

02 На каком этапе проводится проверка

На этапе первичного разбора (префильтрации) – сразу после получения файла и до его отправки в песочницу или дизассемблер

03 Последствия

- Вредоносный файл с измененной структурой может пройти «мимо» системы проверки, но успешно запуститься на компьютере жертвы;
- Файлы придется либо разбирать вручную, либо в большом количестве отправлять в песочницу или дизассемблер, что может быть долгим или накладным по ресурсам
- Намеренно поврежденный файл может вызвать ошибку памяти, зависание или аварийное падение самого анализатора

Проблематика и актуальность темы

04 Сбои анализаторов на нестандартных файлах

Типовые библиотеки разбора ориентируются на строгий стандарт формата PE и часто выдают ошибку при встрече с намеренно искаженными заголовками. Сама ОС Windows такие ошибки игнорирует и успешно запускает программу.

05 Ограничения популярных утилит для автоматизации

Доступные инструменты первичной проверки требуют платную лицензию для работы из консоли и выгрузки отчетов (pestudio), работают только через графическое окно для ручного анализа (PE-bear) или давно не обновляются (Manalyze).

06 Высокая ресурсоемкость глубокого анализа

Проверка файлов в песочницах и тяжелых дизассемблерах занимает десятки секунд. Перед ними необходим быстрый предварительный фильтр, способный за миллисекунды проверить структуру файла и отсеять подозрительные образцы.

Цель и задачи работы

Цель: Разработать открытый инструмент первичного статического анализа PE для выявления аномалий структуры и техник сокрытия данных с учетом поведения загрузчика Windows

[1]	Анализ спецификации PE и обработки заголовков загрузчиком, систематизация техник обхода анализа
[2]	Собственный устойчивый парсер заголовков с контролем границ памяти
[3]	Правила детекции: модификации заголовков, секции W+X, скрытые оверлеи, подделка Rich Header, энтропия секций
[4]	Аналитическое ядро в виде разделяемой библиотеки и CLI с отчетом в JSON
[5]	Тесты на системных файлах и корпусе Corkami, сравнение с аналогами

Объект и предмет исследования

ОБЪЕКТ ИССЛЕДОВАНИЯ

Исполняемые файлы формата PE

Включая образцы с намеренно модифицированной, нестандартной или частично поврежденной структурой заголовков и секций.

Исследуются бинарные файлы x86/x64 архитектур в форматах PE32 и PE32+.

ПРЕДМЕТ ИССЛЕДОВАНИЯ

Методы и алгоритмы статического выявления аномалий

Алгоритмы детекции техник сокрытия данных и расхождений (parser differentials) между средствами статического разбора и загрузчиком ОС Windows.

Фокус на устойчивости алгоритмов верификации к намеренным структурным искажениям.

Открытое ядро для аналитиков и конвейеров обработки

Модульность по аналогии с Suricata

Открытый движок с возможностью добавления пользовательских сигнатур и правил без привязки к закрытой инфраструктуре или платным лицензиям.

Целевые пользователи

Вирусные аналитики, исследователи информационной безопасности, энтузиасты и разработчики автоматизированных конвейеров анализа ВПО.

Пакетная фильтрация на сервере

Предварительная экспресс-сортировка потока входящих бинарных файлов по JSON-вердикту перед передачей в ресурсоемкую песочницу.

Изолированный локальный стенд

Быстрая проверка файлов через CLI на защищенной рабочей машине аналитика без доступа к внешней сети, облачным API и сторонним базам.

Стек технологий и методика тестирования

Компонент	Выбранное решение
Язык разработки	C++20 (или Rust)
Архитектура	Автономный модуль (библиотека + CLI) без внешних зависимостей
Формат вывода	JSON-отчет со списком обнаруженных аномалий и итоговой оценкой риска
Тестовый набор	Системные файлы Windows и верификационный корпус аномальных PE Corkami

Аналоги

Сравнение скорости (мс/файл) и полноты выявления аномалий:

- **pefile** — стандартный python-парсер
- **Manalyze** — статический C++ анализатор
- **Detect It Easy (diec)** — сигнатурный CLI-парсер

Ожидаемые результаты

Разделяемая библиотека (DLL / .so)	Независимое ядро разбора PE и проверки аномалий для встраивания
Консольная утилита (CLI)	Локальный и пакетный запуск, отчет и уровень риска в JSON
Метрики	Скорость (мс/файл) и устойчивость к битым заголовкам в сравнении с аналогами
Репозиторий	**https://github.com/DmitriyKolesnikM8O/diplom**